

Technical Execution Plan: Phases 1 & 2

Underwater 3D Gaussian Splatting Reconstruction

Joshua Hecke

September 7, 2026

1 Introduction

This document outlines the detailed technical steps required to execute Phase 1 (Foundation and Method Selection) and Phase 2 (Baseline Reconstruction and Optimization) of the Scene Change Detection in Underwater 3D Gaussian Splatting project. It specifically addresses two critical constraints:

1. Utilizing a dual-boot Ubuntu setup on a high-performance machine equipped with an NVIDIA RTX 5070 Ti (16GB GDDR7).
2. Leveraging the open-sourced **Submerged3D** dataset for initial testing and validation before accessing the AIMS EcoRRAP dataset.

2 Phase 1: Environment and Hardware Setup

2.1 Ubuntu and NVIDIA Drivers

The machine utilizes an RTX 5070 Ti, which is based on the latest architecture. It is critical to use the latest NVIDIA proprietary drivers.

```
1 # After installing Ubuntu 24.04 (or 22.04), update the system
2 sudo apt update && sudo apt upgrade -y
3
4 # Install standard build essentials
5 sudo apt install build-essential git wget curl -y
6
7 # Add the proprietary graphics drivers PPA and install the latest driver
8 sudo add-apt-repository ppa:graphics-drivers/ppa
9 sudo apt update
10 sudo ubuntu-drivers autoinstall
11 # Reboot the machine after installation
12 sudo reboot
```

2.2 CUDA Toolkit and Conda Configuration

Because the RTX 5070 Ti requires up-to-date software, we will aim to use the latest CUDA versions (e.g., CUDA 12.1+).

Important Note on CUDA Compatibility: Several of the evaluated codebases originally specify **CUDA 11.8**. If you encounter compilation errors (specifically with the **diff-gaussian-rasterization** modules) when using CUDA 12.x, you will need to downgrade the environment's PyTorch and CUDA toolkit to 11.8. Notes are provided in each section below.

```
1 # Download and install Miniconda
2 mkdir -p ~/miniconda3
3 wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O ~/
  miniconda3/miniconda.sh
4 bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
5 rm -rf ~/miniconda3/miniconda.sh
6 ~/miniconda3/bin/conda init bash
7 # Restart terminal
```

2.3 Dataset Preparation (Submerged3D)

We will use the Submerged3D dataset hosted on HuggingFace.

```
1 # Install HuggingFace CLI
2 pip install -U "huggingface_hub[cli]"
3
4 # Download the Submerged3D dataset
5 huggingface-cli download theflash987/Submerged3D --repo-type dataset --local-dir ~/UMP/
  Codebase/Dataset/Submerged3D
```

The Submerged3D dataset contains 4 environments (e.g., Cormoran, Isro, Kwaj, Tokai), each with 20 images in an `images` folder, but no COLMAP data.

2.4 Generating COLMAP Data

The `convert.py` script from the standard 3DGS repository expects input images to be located in an `input/` folder. Because the Submerged3D dataset provides images in an `images/` folder, you must first rename this folder to `input/`.

```
1 # Rename the images directory for each scene
2 mv ~/UMP/Codebase/Dataset/Submerged3D/<scene_name>/images ~/UMP/Codebase/Dataset/
  Submerged3D/<scene_name>/input
```

Then, you can generate the required sparse point cloud and camera poses for 3DGS using the `convert.py` script:

```
1 # Install COLMAP (and FFmpeg if needed)
2 sudo apt install colmap ffmpeg -y
3
4 # The original 3D Gaussian Splatting repository is already provided in the Tools
  directory.
5 cd ~/UMP/Codebase/Tools/gaussian-splatting-main
6
7 # Create a temporary conda environment to run the script
8 conda create -n colmap_runner python=3.9 -y
9 conda activate colmap_runner
10
11 # Run the convert script on each of the 4 scenes
12 # Replace <scene_name> with Cormoran, Isro, Kwaj, and Tokai
13 python convert.py -s ~/UMP/Codebase/Dataset/Submerged3D/<scene_name>
```

This will create a `sparse/0` folder inside each scene directory, containing the `cameras.bin`, `images.bin`, and `points3D.bin` files required by the 3DGS approaches. The script will also create an `images` folder containing undistorted versions of the inputs.

2.5 Depth Map Generation and Frame Interpolation

Several models (3D-UIR, UW-GS, and RUSplatting) strictly require or highly recommend depth maps. RUSplatting additionally requires inverted depth maps and heavily relies on interpolated frames due to the sparse views (only 20 images per scene).

1. Depth Map Generation (Depth-Anything-V2)

```
1 cd ~/UMP/Codebase/Tools/Depth-Anything-V2-main
2 conda create -n depth_anything python=3.10 -y
3 conda activate depth_anything
4 pip install torch torchvision torchaudio
5 pip install -r requirements.txt
6
7 # Download the weights manually
8 mkdir checkpoints
9 wget "https://huggingface.co/depth-anything/Depth-Anything-V2-Large/resolve/main/
  depth_anything_v2_vitl.pth?download=true" -O checkpoints/depth_anything_v2_vitl.pth
10
11 # For standard depth maps (e.g., for 3D-UIR and UW-GS):
12 python run.py --encoder vitl --pred-only --grayscale --img-path ~/UMP/Codebase/Dataset/
  Submerged3D/<scene_name>/images --outdir ~/UMP/Codebase/Dataset/Submerged3D/<
  scene_name>/depthmap
13
14 # For inverted depth maps (Required by RUSplatting):
15 # Note: You MUST modify run.py line 63 to include 'depth = 255.0 - depth' before running
  this.
```

```

16 python run.py --encoder vitl --pred-only --grayscale --img-path ~/UMP/Codebase/Dataset/
    Submerged3D/<scene_name>/images --outdir ~/UMP/Codebase/Dataset/Submerged3D/<
    scene_name>/depthmap_inverted

```

2. Frame Interpolation (RIFE) - Required for RUSplatting RUSplatting requires generating intermediate frames between your 20 source images (named with a `_to_` flag, e.g., `frame000_to_frame001.jpg`). You will need to utilize a tool like RIFE (Real-Time Intermediate Flow Estimation) to generate these prior to running RUSplatting.

3 Phase 2: Baseline Reconstruction and Optimization

We will train 6 different models. For each model, assume you are running the commands from the root of the respective repository inside `/UMP/Codebase/3DGS-Water-Approaches/`.

Note on Submodules: Before installing dependencies for any model, run `git submodule update --init --recursive` inside the repository to ensure all submodules are properly downloaded.

3.1 Model 1: SeaSplat

```

1 cd ~/UMP/Codebase/3DGS-Water-Approaches/Physics/seasplat-master
2 conda create --name seasplat_py310 -y python=3.10
3 conda activate seasplat_py310
4
5 # Try latest CUDA first. If it fails, use: pip install torch==2.1.0 torchvision==0.16.0
    torchaudio==2.1.0 --index-url https://download.pytorch.org/whl/cu118
6 pip install torch torchvision torchaudio
7 pip install -r requirements.txt
8 pip install submodules/diff-gaussian-rasterization
9 pip install submodules/simple-knn
10
11 # Train
12 python train.py -s ~/UMP/Codebase/Dataset/Submerged3D/<scene_name> --exp seasplat_exp --
    do_seathru --seathru_from_iter 10000 --eval

```

3.2 Model 2: 3D-UIR

```

1 cd ~/UMP/Codebase/3DGS-Water-Approaches/Physics/3D-UIR-main
2 conda create -n 3d-uir python=3.10
3 conda activate 3d-uir
4
5 # NOTE: Originally uses CUDA 11.8. Try latest first. If failure:
6 # pip install torch==2.1.0 torchvision==0.16.0 torchaudio==2.1.0 --index-url https://
    download.pytorch.org/whl/cu118
7 # conda install cudatoolkit-dev=11.8 -c conda-forge
8 pip install torch torchvision torchaudio
9
10 pip install submodules/diff-gaussian-rasterization
11 pip install submodules/simple-knn/
12 pip install submodules/fused-ssim/
13 pip install ninja git+https://github.com/NVlabs/tiny-cuda-nn/#subdirectory=bindings/
    torch
14
15 # 3D-UIR expects the depth directory to be named "depths" instead of "depthmap"
16 # It also requires a depth parameters JSON file
17 cd ~/UMP/Codebase/Dataset/Submerged3D/<scene_name>
18 ln -s depthmap depths
19 cd ~/UMP/Codebase/3DGS-Water-Approaches/Physics/3D-UIR-main
20
21 # Generate depth parameters for 3D-UIR
22 python utils/make_depth_scale.py --base_dir ~/UMP/Codebase/Dataset/Submerged3D/<
    scene_name> --depths_dir ~/UMP/Codebase/Dataset/Submerged3D/<scene_name>/depths
23
24 # Train
25 python train.py -s ~/UMP/Codebase/Dataset/Submerged3D/<scene_name> -d ~/UMP/Codebase/
    Dataset/Submerged3D/<scene_name>/depths --eval

```

3.3 Model 3: Gaussian Splashing

```
1 cd ~/UMP/Codebase/3DGS-Water-Approaches/Image/gaussianSplashing-main
2 conda env create --file environment.yml
3 conda activate gaussianSplashing_env
4
5 # Train using Hybrid mode
6 python train.py -s ~/UMP/Codebase/Dataset/Submerged3D/<scene_name> --
   underwater_processing HYB --eval
```

3.4 Model 4: WaterSplatting

```
1 cd ~/UMP/Codebase/3DGS-Water-Approaches/Image/water-splatting-main
2 conda create --name water_splatting -y python=3.8
3 conda activate water_splatting
4
5 # NOTE: Originally built for CUDA 11.8.
6 pip install torch==2.1.2+cu118 torchvision==0.16.2+cu118 --extra-index-url https://
   download.pytorch.org/whl/cu118
7 conda install -c "nvidia/label/cuda-11.8.0" cuda-toolkit
8 pip install ninja git+https://github.com/NVlabs/tiny-cuda-nn/#subdirectory=bindings/
   torch
9 pip install nerfstudio==1.1.4
10 ns-install-cli
11 pip install --no-use-pep517 -e .
12
13 # Train
14 ns-train water-splatting --vis viewer+wandb colmap --downscale-factor 1 --colmap-path
   sparse/0 --data ~/UMP/Codebase/Dataset/Submerged3D/<scene_name> --images-path images
```

3.5 Model 5: RUSplatting

```
1 cd ~/UMP/Codebase/3DGS-Water-Approaches/Additional/RUSplatting-main
2 conda create -n rusplatting python=3.12
3 conda activate rusplatting
4
5 # Uses PyTorch 2.5.1 and CUDA 12.8 natively
6 pip install torch torchvision torchaudio
7 pip install submodules/diff-gaussian-rasterization
8 pip install submodules/simple-knn
9
10 # RUSplatting expects the inverted depth maps to be available in "depthmap"
11 cd ~/UMP/Codebase/Dataset/Submerged3D/<scene_name>
12 mv depthmap depthmap_standard
13 ln -s depthmap_inverted depthmap
14 cd ~/UMP/Codebase/3DGS-Water-Approaches/Additional/RUSplatting-main
15
16 # Train
17 python train.py -s ~/UMP/Codebase/Dataset/Submerged3D/<scene_name> --adaptive --eval
18
19 # Restore original depthmap after training
20 cd ~/UMP/Codebase/Dataset/Submerged3D/<scene_name>
21 rm depthmap
22 mv depthmap_standard depthmap
```

3.6 Model 6: UW-GS

```
1 cd ~/UMP/Codebase/3DGS-Water-Approaches/Additional/UW-GS-main
2 conda env create -f environment.yml
3 conda activate UW-GS
4
5 # NOTE: Environment targets Python 3.7, PyTorch 1.12.1, CUDA 11.6. It is highly
   recommended to follow these versions strictly for this codebase.
6 pip install imageio==2.27.0 opencv-python imageio-ffmpeg scipy dearpygui lpips
7
8 # Train
9 python train.py -s ~/UMP/Codebase/Dataset/Submerged3D/<scene_name> -m output/scene --
   BMM_Flag --eval
```

4 Mesh Extraction with SuGaR (The 7th Splat)

Once the 6 models are trained, evaluate their output point clouds visually. Select the output directory of the model that provides the cleanest water-removed geometry. We will use this pre-trained 3DGS model as a prior for SuGaR.

```
1 cd ~/UMP/Codebase/3DGS-Water-Approaches/Additional/SuGaR-main
2 conda create --name sugar -y python=3.9
3 conda activate sugar
4
5 # NOTE: Requires CUDA 11.8 for compatibility with PyTorch3D and its specific rasterizer
  versions.
6 conda install pytorch==2.0.1 torchvision==0.15.2 torchaudio==2.0.2 pytorch-cuda=11.8 -c
  pytorch -c nvidia
7 conda install -c fvcore -c iopath -c conda-forge fvcore iopath
8 conda install pytorch3d==0.7.4 -c pytorch3d
9 conda install -c plotly plotly
10 conda install -c conda-forge rich plyfile==0.8.1 jupyterlab nodejs ipywidgets
11 pip install open3d PyMCubes
12
13 pip install -e gaussian_splatting/submodules/diff-gaussian-rasterization/
14 pip install -e gaussian_splatting/submodules/simple-knn/
15
16 # Run SuGaR on the best pre-trained model
17 python train_full_pipeline.py -s ~/UMP/Codebase/Dataset/Submerged3D/<scene_name> -r "
  dn_consistency" --high_poly True --export_obj True --gs_output_dir <
  path_to_best_model_output>
```

This will generate a highly detailed, surface-aligned 3D Gaussian Splatting model and extract a textured .obj mesh representing the clear-water baseline structure.

5 Evaluation and Quantitative Metrics

To produce a quantitative table comparing the models (e.g., PSNR, SSIM, LPIPS), it is critical that the models are trained with the `--eval` flag (as specified in the commands above). This flag holds out a subset of images (usually every 8th image) for testing.

5.1 Standard 3DGS-based Models

For SeaSplat, 3D-UIR, Gaussian Splashing, RUSplatting, and UW-GS, evaluation is a two-step process using the scripts provided in the respective forks:

```
1 # Step 1: Render the test set
2 # Run this from the root of the specific model's repository
3 python render.py -m <path_to_model_output> --skip_train
4
5 # Step 2: Compute metrics
6 python metrics.py -m <path_to_model_output>
```

The output will be saved in `<path_to_model_output>/results.json`, detailing the PSNR, SSIM, and LPIPS scores on the holdout set.

5.2 WaterSplatting (Nerfstudio)

WaterSplatting automatically computes metrics during training and evaluates on a holdout set by default. You can view the final metrics in the console output or the WandB dashboard (if configured). Alternatively, you can explicitly evaluate a trained model by running:

```
1 ns-eval --load-config <path_to_outputs>/config.yml
```

5.3 Viewing Outputs Visually

To visually inspect the trained models:

- **Standard 3DGS Models:** Use the pre-compiled **SIBR Viewers** provided by the original 3D Gaussian Splatting repository. Alternatively, you can drag and drop the output `point_cloud.ply`

(located in `<path_to_model_output>/point_cloud/iteration_30000/`) into a WebGL viewer like SuperSplat or Antimatter15's Splat Viewer.

- **WaterSplatting (Nerfstudio):** Run `ns-viewer --load-config <path_to_outputs>/config.yml` and open the provided localhost link in your browser.